SHACL: проверка данных по формам

Обучающая презентация к лекции 3: от первой формы к запросам

О. М. Атаева А. П. Халов

Осень 2026

Два вопроса к записанному знанию

Что из записанного следует?

- Отвечает машина вывода.
- Открытый мир: неизвестное не считается ложным.
- «У рецензии нет статьи» означает: статья есть, пока не названа.
- Результат: новые факты, классификация, объяснения.

Всё ли записано и верно ли?

- Отвечает проверка данных по формам.
- Замкнутый мир: чего нет в графе, того нет.
- «У рецензии нет статьи» означает: нарушение.
- Результат: отчёт о несоответствиях, граф не меняется.

SHACL, Shapes Constraint Language, язык ограничений в виде форм. Рекомендация W3C 2017 года. Описывает, как должны выглядеть данные, и выдаёт отчёт о каждом месте, где они выглядят иначе.

Главное

Машина вывода отвечает, что следует из записанного. Форма отвечает, всё ли записано и верно ли. Одно средство другое не заменяет.

Три участника




```
pyshacl -s shapes.ttl data.ttl
```

```
from pyshacl import validate
ok, report_graph, report_text = validate(data, shacl_graph=shapes, inference="none")
```

Главное

Все три участника суть графы RDF. Формы пишутся на том же языке, что и данные, и могут храниться рядом с ними.

Учебный граф: четыре книги, три автора

```
:к1 a :Книга ; :название "Логика" ;    :автор :Тарский ; :год 1936 .
:к2 a :Книга ; :название "Онтологии" ; :автор :Лами ; :автор :Груббер ; :год 2021 .
:к3 a :Книга ;                          :автор :Поппер ; :год "1972"^^xsd:string .
:к4 a :Книга ; :название "Знание" ;                      :год 1945 .
:Тарский a :Автор ; :имя "Альфред Тарский" .
:Лами a :Автор .   :Поппер a :Автор .
```

Четыре подвоха, по одному на форму:

- у книги к3 нет названия,
- у книги к4 нет автора,
- год книги к3 записан строкой,
- Груббер нигде не объявлен автором.

Главное

Формы будут находить подвохи по одному. Каждый пример ниже прогнан pySHACL, отчёты настоящие.

Форма с одним ограничением

```
shp:Книга a sh:NodeShape ;
    sh:targetClass :Книга ;
    sh:property [
        sh:path :название ;
        sh:minCount 1 ;
        sh:message "у книги нет названия" ] .
```

```
Conforms: False
  к3  название  у книги нет названия
      Violation  MinCountConstraintComponent
```

- `sh:NodeShape`: описание того, как должен выглядеть узел.
- `sh:targetClass`: цель, проверке подлежат все узлы с типом Книга.
- `sh:property`: одно ограничение на свойство: по пути `:название` не меньше одного значения.
- `sh:message`: текст для отчёта.

Что делает форма: берёт каждую книгу, считает значения названия, если их нет, пишет строку в отчёт.

Главное

Строка отчёта содержит пять вещей: узел, путь, сообщение, строгость, сработавшее ограничение. Этого достаточно, чтобы найти и исправить.

Число связей и тип значения

Сколько значений

```
sh:property [ sh:path :автор ; sh:maxCount 1 ;
    sh:severity sh:Warning ;
    sh:message "больше одного автора" ] .
```

```
  к2  автор  больше одного автора  Warning
```

`sh:minCount` не меньше, `sh:maxCount` не больше, вместе «ровно».

Какое значение

```
sh:property [ sh:path :автор ; sh:class :Автор ;
    sh:minCount 1 ;
    sh:message "автор записан и он автор" ] .
```

```
  к2  автор  ...  ClassConstraintComponent
  к4  автор  ...  MinCountConstraintComponent
```

У к4 автора нет. У к2 Груббер записан, но тройки «Груббер а Автор» в графе нет. Форма не рассуждает, кем он мог бы быть.

`sh:class` значение есть индивид класса, `sh:datatype` литерал типа, `sh:nodeKind` вид узла: IRI, литерал, пустой узел.

Главное

Замкнутый мир в действии: форма смотрит, что записано. Отсутствующий тип у Груббера есть нарушение.

Допустимые значения и строгость

```
sh:property [ sh:path :год ;
    sh:datatype xsd:integer ;
    sh:minInclusive 1450 ;
    sh:message "год: целое не раньше 1450" ] .

  к3  год  ...  DatatypeConstraintComponent
  к3  год  ...  MinInclusiveConstraintComponent
```

Строка «1972» проваливает обе проверки. Отчёт перечисляет каждое сработавшее ограничение отдельно.

ограничение	смысл
`sh:in (5 10)`	значение из списка
`sh:hasValue true`	среди значений есть это
`sh:minInclusive`, `sh:maxInclusive`	диапазон
`sh:pattern "^[А-ЯЁ]"`	регулярное выражение

Строгость `sh:severity`: Violation (по умолчанию), Warning, Info. Пометка в отчёте, ни на что больше не влияет.

Главное

Итог `Conforms` истинен только при пустом отчёте. Предупреждение тоже результат: данные с одним предупреждением получают `Conforms: False`.

Журнал рецензий: три проверки записи

```
shp:Рецензия a sh:NodeShape ;
  sh:targetClass :Рецензия ;
  sh:property [ sh:path :на_статью ; sh:class :Статья ;
      sh:minCount 1 ; sh:maxCount 1 ;
      sh:message "у рецензии ровно одна статья" ] ;
  sh:property [ sh:path :шкала ; sh:in (5 10) ;
      sh:message "шкала 5 или 10" ] ;
  sh:property [ sh:path :оценка ; sh:datatype xsd:decimal ;
      sh:minInclusive 1 ; sh:maxInclusive 10 ;
      sh:message "оценка от 1 до 10" ] .

Conforms: True
```

- Онтология примера редактора `review.owl`: статьи P1–P8, четырнадцать рецензий, рецензенты, положение.
- Три ограничения в одной форме, у каждого своё сообщение.
- На чистых данных все четырнадцать рецензий проходят.
- Ловушка: оценка в файле записана как `7.0` с типом `xsd:decimal`. Форма с `xsd:integer` даст тринадцать нарушений.

Главное

Первая ошибка агента из лекции 1, усреднение по разным шкалам, здесь ловится формой: шкала записана и проверена.

Сравнение значений двух свойств

```
sh:property [ sh:path :назначено ;
    sh:lessThanOrEquals :сдано ;
    sh:message "назначено позже сдачи" ] .
```

Для каждой рецензии каждая дата назначения сравнивается с каждой датой сдачи. У P8 даты сдачи нет, сравнивать не с чем, нарушения нет.

```
sh:property [ sh:path :сдано ;
    sh:lessThanOrEquals :срок ;
    sh:message "6.6: сдана позже срока" ] .
```

```
  рец_P5_Rv3  сдано  6.6: сдана позже срока
```

- `sh:lessThan`, `sh:lessThanOrEquals`, `sh:equals`, `sh:disjoint`: сравнение значений двух свойств одного узла. В OWL этого нет.
- Требование 6.6 «рецензия в 21 день» нуждается в арифметике дат, которой в ядре SHACL нет. Свойство `срок` вычислено при загрузке: назначение плюс 21 день.
- Перепутанные стороны дают тринадцать нарушений: у каждой сданной рецензии сдача позже назначения.

Главное

Разделение: вычисление делается в коде, проверка записана в данных. Вторая дорога, арифметика внутри запроса, дальше.

Обратный путь: требование 6.1

Попытка новичка

```
shp:Статья a sh:NodeShape ; sh:targetClass :Статья ;
  sh:property [ sh:path :имеет_рецензию ;
      sh:minCount 2 ; sh:message "6.1" ] .

  P1 ... P8   имеет_рецензию   6.1   (8 нарушений)
```

Свойство `имеет_рецензию` машина вывода лекции 2 выводила, в файле его нет ни у кого. Форма видит только записанное.

Правильная запись

```
sh:property [
  sh:path [ sh:inversePath :на_статью ] ;
  sh:qualifiedValueShape [ sh:property [
      sh:path :сдано ; sh:minCount 1 ] ] ;
  sh:qualifiedMinCount 2 ;
  sh:message "6.1: сданных рецензий меньше двух" ] .

  P4   6.1: сданных рецензий меньше двух
  P8   6.1: сданных рецензий меньше двух
```

Обратный путь: от статьи к рецензиям, которые ссылаются на неё. Считаются только сданные.

Главное

Выведенное машиной вывода форма не видит. Связь либо переписывается через `sh:inversePath`, либо материализуется до проверки.

Вложенные формы и логика

`sh:node`: значение должно пройти другую форму

```
shp:Книга a sh:NodeShape ; sh:targetClass :Книга ;
  sh:property [ sh:path :автор ; sh:node shp:Автор ] .
shp:Автор a sh:NodeShape ;
  sh:property [ sh:path :имя ; sh:minCount 1 ;
      sh:message "у автора нет имени" ] .

  Груббер  имя    у автора нет имени
  Лами     имя    у автора нет имени
  Поппер   имя    у автора нет имени
  к2       автор  Value does not conform to shp:Автор
```

Отчёт двухъярусный: внутренняя форма называет автора, внешняя называет книгу. У `shp:Автор` нет цели: она применяется только там, куда её позвали.

`sh:or`: хотя бы одна из форм

```
shp:Рецензия a sh:NodeShape ;
  sh:targetClass :Рецензия ;
  sh:or ( [ sh:property [ sh:path :оценка ;
                          sh:minCount 1 ] ]
          [ sh:property [ sh:path :сдано ;
                          sh:maxCount 0 ] ] ) ;
  sh:message "сданная рецензия без оценки" .
```

«Либо есть оценка, либо рецензия не сдана». Несданная P8 проходит по второй ветви. Есть также `sh:and`, `sh:not`, `sh:xone`.

Путь из двух шагов (`:рецензент :из_организации`): от рецензии к организации рецензента.

Главное

Формы соединяются: вложением, подсчётом подходящих значений, логическими связками. Путь может быть обратным и составным.

Цели и закрытые формы

цель	какие узлы проверяются
`sh:targetClass :Книга`	все книги и экземпляры подклассов
`sh:targetNode :P3`	один названный узел
`sh:targetSubjectsOf :автор`	все, у кого есть свойство автор
`sh:targetObjectsOf :автор`	все значения свойства автор

Цель по объектам свойства находит Груббера: у него нет типа, под цель по классу он не попал бы. Подклассы цель по классу захватывает по записанным `rdfs:subClassOf`, без машины вывода.

Закрытая форма

```
shp:Книга a sh:NodeShape ; sh:targetClass :Книга ;
  sh:closed true ;
  sh:ignoredProperties ( rdf:type ) ;
  sh:property [ sh:path :название ] ;
  sh:property [ sh:path :автор ] .

  к1 ... к4   год   ClosedConstraintComponent
```

Разрешены только свойства из формы и из списка исключений. У всех книг есть `год`, которого в списке нет. Нужна для приёма данных по строгой схеме.

Главное

Обычная форма безразлична к лишним свойствам. Закрытая форма замечает опечатку в имени свойства.

Когда ядра не хватает: требование 6.2

```
<http://example.org/shapes> a owl:Ontology ;
  sh:declare [ sh:prefix "" ;
    sh:namespace "http://example.org/review.owl#"^^xsd:anyURI ] .

shp:Рецензия a sh:NodeShape ; sh:targetClass :Рецензия ;
  sh:sparql [
    sh:message "6.2: рецензент из организации авторов" ;
    sh:prefixes <http://example.org/shapes> ;
    sh:select """
      SELECT $this WHERE {
        $this :рецензент ?p .  ?p :из_организации ?o .
        $this :на_статью ?s .  ?s :организация_авторов ?o . }
    """ ] .

  рец_P3_Rv4   6.2: рецензент из организации авторов
```

- Нужно сравнить организацию по пути рецензия → рецензент → организация с организацией по пути рецензия → статья → авторы. Ядро сравнивает значения двух свойств одного узла, здесь два разных пути.
- `sh:sparql`: ограничение запросом. `$this` есть проверяемый узел. Запрос возвращает нарушителей.
- Те же четыре условия, что в правиле 6.2 лекции 2. Правило добавляет факт, форма добавляет строку отчёта.

Главное

Тем же способом записывается 6.6 без свойства срок: `FILTER (?сдано > ?назначено + "P21D"^^xsd:dayTimeDuration)`. Арифметика дат есть в SPARQL.

SHACL и OWL рядом

	OWL и машина вывода	SHACL
вопрос	что следует из записанного	всё ли записано и верно ли
мир	открытый	замкнутый
результат	новые факты, классификация, объяснение	отчёт, граф не меняется
«у рецензии нет статьи»	статья есть, пока не названа	нарушение minCount
«рецензент не из организации авторов»	правило выводит факт «нарушает»	запрос даёт строку отчёта

Одно требование положения получает разные записи: правило, когда нужен новый факт, выражение класса, когда нужна классификация, форма, когда нужен отчёт о данных, запрос, когда нужен список.

Главное

Форма видит только записанное. Выведенные связи либо переписываются в путях, либо записываются в граф до проверки.

Типичные ошибки и шпаргалка

Ошибки

- форма на выведенное свойство без `sh:inversePath`,
- не тот тип литерала: `integer` вместо `decimal`,
- перепутанные стороны сравнения,
- `sh:pattern` без `sh:minCount`: отсутствие не замечено,
- сообщение без пункта требования,
- ожидание `Conforms: True` при предупреждении,
- намеренная ошибка в сдаваемом файле.

Шпаргалка

число	`minCount`, `maxCount`
тип	`class`, `datatype`, `nodeKind`
значения	`in`, `hasValue`, `minInclusive`, `pattern`
сравнение	`lessThanOrEquals`, `equals`, `disjoint`
пути	`inversePath`, `( :p :q )`, `zeroOrMorePath`
составные	`node`, `qualifiedValueShape`, `or`, `not`
цели	`targetNode`, `targetObjectsOf`
запрос	`sh:sparql`

Главное

Подробный конспект с теми же примерами и задачами: `SHACL_для_начинающих.pdf`.